

Voice Pipelines Optimized for the Wrong Metric

She said “check my balance” and the ASR heard “check my balance” perfectly. Word error rate: zero. The pipeline routed her to billing.

This is the failure mode no one talks about. Not because it’s rare — because it’s invisible. The transcript was correct. The logs looked clean. The customer got transferred to the wrong department, waited nine minutes, explained her problem again, and the CSAT score dropped. Somewhere in a dashboard, WER sat at 4.2% and an engineering team congratulated themselves.

What Actually Breaks

The voice pipeline industry has spent a decade optimizing for transcription fidelity. WER — word error rate — is the metric that launches papers, wins benchmarks, and gets printed on vendor datasheets. And it measures exactly the wrong thing.

WER asks: *did we get the words right?*

It does not ask: *did we understand what the caller wanted?*

These are different questions with different failure modes. A pipeline can achieve perfect WER and still misclassify intent on 15% of calls. The words are right. The meaning is wrong.

Here’s what silent intent misclassification looks like in production:

- **The transcript is correct.** “I want to cancel my subscription” comes through clean.
- **The intent classifier maps it wrong.** It routes to “billing inquiry” instead of “cancellation.”
- **No error is raised.** The system behaved as designed — it classified with high confidence.
- **No eval loop catches it.** There is no eval loop. The pipeline was configured once, tested on twelve examples, and deployed.
- **No improvement signal exists.** Without systematic evaluation, there is no data to tell you *which* intents are confused, *which* utterance patterns trigger misclassification, or *which* pipeline component is responsible.

The failure is not in the ASR. The failure is downstream, in the intent extraction layer, and it compounds silently because nothing in the pipeline is designed to detect it.

Why Better ASR Is Not Enough

The instinct is to upgrade the ASR model. Get a better transcription engine. Reduce WER from 4.2% to 2.8%. This is the wrong lever.

Consider two metrics:

WER (Word Error Rate): The edit distance between the transcript and the reference text, normalized by reference length. Measures transcription fidelity.

IER (Intent Error Rate): The fraction of utterances where the predicted intent does not match the expected intent. Measures task completion.

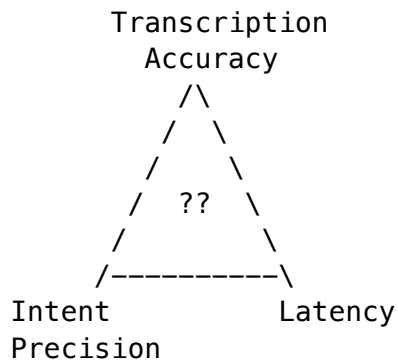
WER optimizes for acoustic-linguistic accuracy. IER optimizes for semantic correctness. A pipeline can have high WER and low IER (the transcript is messy but the intent is clear). A pipeline can have low WER and high IER (the transcript is perfect but the classifier is confused).

The relationship between these metrics is not monotonic. Improving WER does not reliably improve IER. In many cases, the ASR is already good enough — the errors live in the normalizer, the classifier prompt, the confidence threshold, the decision policy. But because we only measure WER, we never look there.

This is not a tooling gap. It's a measurement gap. And measurement gaps don't fix themselves.

The Tradeoff You Don't Know You're Making

Every voice pipeline navigates a tradeoff triangle, whether the team acknowledges it or not:



Most teams optimize one vertex — transcription accuracy — and treat the other two as fixed. But they're not fixed. They're coupled.

- **Transcription Accuracy vs. Intent Precision:** Better transcription doesn't guarantee better intent. A normalizer that strips filler words ("um", "uh", "like") improves classifier performance but changes the transcript. An n-best ASR list improves intent precision by giving the classifier multiple hypotheses, but the "best" transcript by WER is not always the best transcript for intent.
- **Intent Precision vs. Latency:** Running a reranker over candidate intents improves precision but adds a round-trip. A decision policy that abstains when confidence is low improves precision but requires a fallback path. Every precision gain has a latency cost.
- **Latency vs. Transcription Accuracy:** Streaming ASR with aggressive endpointing is fast but misses trailing words. Batch processing with full-utterance context is accurate but slow. The latency budget constrains the accuracy ceiling.

The problem is not that teams make the wrong tradeoff. The problem is that teams don't know they're making a tradeoff, because they're only measuring one axis.

The Measurement Gap in Practice

Here's what the measurement gap looks like operationally:

1. **No eval suite.** Most voice pipelines in production have no systematic evaluation beyond "try a few utterances and see if they work." There is no holdout set. There is no phenomenon coverage. There is no regression testing.
2. **No attribution.** When an intent is misclassified, there is no trace showing *which component* caused the failure. Was it the ASR? The normalizer? The classifier prompt? The confidence

threshold? Without component-level tracing, debugging is guesswork.

3. **No improvement loop.** Even teams that do spot-check their pipeline have no mechanism to *systematically* improve it. They find a failure, fix the prompt, break something else, and iterate by hand until the next fire drill.

The result is a pipeline that is exactly as good as the day it was deployed. It does not learn from failures. It does not adapt to new intents. It does not even know what it's getting wrong.

What This Actually Costs

The cost is not abstract. In a customer service voice pipeline handling 10,000 calls per day:

- A 10% IER means 1,000 misrouted calls daily
- Each misroute adds 3-5 minutes of handle time (transfer, re-explain, re-route)
- At \$1/minute agent cost, that's \$3,000-\$5,000/day in wasted agent time
- CSAT drops 15-20 points on misrouted calls
- And the pipeline team doesn't know it's happening, because WER looks fine

The fix is not a better ASR model. The fix is measuring the right thing, building an eval loop that catches silent failures, and creating an improvement mechanism that operates at the intent level — not the word level.

That's what this system does. Not better transcription. Better understanding. And the infrastructure to know the difference.

A Pipeline That Evaluates Itself

Most voice pipelines are configured once and prayed over. A product manager writes an intent prompt. An engineer picks an ASR provider. Someone tests it with “check my balance” and “transfer me to an agent,” watches both classify correctly, and ships it. The pipeline is now in production, untested against filler words, homophones, out-of-domain utterances, and the thousand ways real humans say the same thing differently.

What if the pipeline generated its own eval suite, ran it, and fixed what broke?

The Core Mechanism

The system uses a multi-agent build loop. When you create a pipeline, you provide an intent prompt — a natural language description of what your callers might want. From that single prompt, the system:

1. **Parses intents** into a structured schema with definitions, examples, disambiguation rules, and out-of-scope boundaries
2. **Generates a diverse eval dataset** covering direct utterances, natural language variants, filler-word patterns, polite forms, and out-of-domain examples — split into train, dev, and immutable holdout sets
3. **Builds a component graph** — ASR □ Normalizer □ Intent Classifier □ Candidate Reranker □ Decision Policy — with each component independently configurable

4. **Runs the eval suite** against the pipeline, measuring IER per split, per intent, with a full confusion matrix
5. **Analyzes failures**, clusters them by confused intent pairs, generates adversarial examples targeting the failure modes, and proposes component-level fixes
6. **Iterates** until IER meets the target threshold or the iteration budget is exhausted

This is not prompt-and-pray. This is build-measure-fix, automated.

Artifact-Driven, Not Prompt-Driven

Every agent in the system produces typed, versioned artifacts — not loose JSON, not unstructured text, not “whatever Claude said.” Each artifact is a Pydantic model with explicit fields, refs to upstream artifacts, and a schema that the next agent in the chain can validate against.

```
class IntentSchemaArtifact(BaseModel):
    artifact_id: str
    pipeline_id: str
    source_prompt: str
    fallback_intent: str = "unknown"
    intents: list[IntentDefinition]

class IntentDefinition(BaseModel):
    intent_name: str
    description: str
    positive_examples: list[str]
    negative_examples: list[str]
    disambiguation_rules: list[str]
    out_of_scope_examples: list[str]
```

Five artifact types flow through the build loop:

Artifact	What It Contains	Produced By
IntentSchemaArtifact	Parsed intents with definitions, examples, disambiguation rules	Schema Parser
EvalDatasetArtifact	Test utterances with splits, phenomenon tags, confusable-with annotations	Eval Curator Agent
PipelineGraphArtifact	Component specs, edges, configuration for every pipeline stage	ETL Pipeline Agent
EvaluationReportArtifact	IER, confusion matrix, per-intent accuracy, hard cases with traces	Eval Runner
AdversarialFindingsArtifact	Failure clusters, proposed adversarial examples, component fix recommendations	Adversarial Agent

Each artifact carries an `ArtifactRef` — a pointer with `artifact_id`, `artifact_type`, `pipeline_id`, and `version`. Artifacts reference their inputs explicitly. The `EvalDatasetArtifact` knows which `IntentSchemaArtifact` it was generated from. The `EvaluationReport`—

tArtifact knows which PipelineGraphArtifact and EvalDatasetArtifact it evaluated. The AdversarialFindingsArtifact knows which EvaluationReportArtifact it analyzed.

```
class ArtifactRef(BaseModel):
    artifact_id: str
    artifact_type: ArtifactType
    pipeline_id: str
    version: int
```

This is not incidental. Artifact lineage is what makes the improvement loop debuggable. When IER drops on iteration 3, you can trace backward: which adversarial examples were added? Which component config changed? Which failure cluster triggered the change? Without typed artifacts and explicit refs, the build loop is a black box. With them, it's an audit trail.

The Component Graph

The pipeline is not a monolith. It's a directed graph of independently tunable components:

```
graph LR
    A[ASR Provider] -->|raw transcript| B[Transcript Normalizer]
    B -->|cleaned text| C[Intent Classifier]
    C -->|candidate intents| D[Candidate Reranker]
    D -->|ranked intents| E[Decision Policy]
    E -->|final intent + confidence| F[Output]
```

Each component has its own spec:

ASR Component — Provider, model, language, keyword hints, n-best count, endpointing config. The keyword_hints field is critical: domain-specific terms that the ASR would otherwise miss. These get updated by the adversarial agent when misclassifications trace back to transcription errors.

```
class AsrComponentSpec(BaseModel):
    component_id: str
    provider: str
    model: str
    keyword_hints: list[str]
    n_best: int = 1
    endpointing_config: dict[str, Any]
```

Transcript Normalizer — Lowercase, strip fillers, punctuation policy, canonical replacements. The normalizer is where “gonna” becomes “going to” and “uhh” gets removed. These rules directly affect classifier performance and are tuned based on eval results.

Intent Classifier — Provider, model, system prompt, candidate count, few-shot examples, and a reference to the intent schema. The system prompt is the primary lever the ETL agent uses to improve IER — rewriting it based on failure analysis.

Candidate Reranker — Takes the top-k candidates from the classifier and applies a separate reranking prompt. This is where the system resolves near-ties between confusable intents.

Decision Policy — Confidence threshold, margin threshold (minimum gap between top-1 and top-2), fallback intent, abstain behavior. This is where the system decides “I’m not sure enough” and

routes to a human.

```
class DecisionPolicyComponentSpec(BaseModel):
    confidence_threshold: float = 0.7
    margin_threshold: float = 0.05
    fallback_intent: str = "unknown"
    allow_abstain: bool = True
    out_of_domain_strategy: str = "fallback_intent"
```

The edges between components are explicit:

```
class PipelineEdge(BaseModel):
    from_component_id: str
    from_output_key: str
    to_component_id: str
    to_input_key: str
```

Why does the graph structure matter? Because when the adversarial agent finds a failure cluster — say, “cancel_subscription” confused with “billing_inquiry” — it needs to know *which component to fix*. If the ASR is garbling the word “cancel,” the fix is keyword_hints. If the classifier prompt doesn’t distinguish the two intents clearly enough, the fix is system_prompt. If the decision policy is routing low-confidence classifications instead of abstaining, the fix is confidence_threshold. Component isolation makes targeted fixes possible.

What the Build Loop Actually Looks Like

```
graph TD
    P[Intent Prompt] --> S[Parse Intent Schema]
    S --> C[Eval Curator Agent]
    C --> G[Build Pipeline Graph]
    G --> E[Run Eval Suite]
    E --> R{IER ≤ target?}
    R -->|Yes| D[Pipeline Ready]
    R -->|No| A[Adversarial Agent]
    A --> O[ETL Pipeline Agent]
    O --> E
```

The orchestrator drives this loop:

```
for iteration in range(max_iters):
    current_ier = context.metrics.get("ier", 1.0)
    if current_ier <= ier_target:
        break

    # Adversarial agent red-teams via tool loop
    adv_result = await self._adversarial_agent.run(context)

    # Re-optimize with expanded eval set
    etl_result = await self._etl_agent.run(context)
```

Each iteration: measure □ red-team □ fix □ re-measure. The adversarial agent adds hard

examples. The ETL agent rewrites the classifier prompt. The eval suite runs again. IER either drops or it doesn't. If it doesn't drop after `max_iters`, the system stops and reports what it couldn't fix — including which failure clusters resisted improvement.

What This Is Not

This is not AutoML for voice. The components are not swapped automatically from some search space. The architecture is fixed — ASR → Normalizer → Classifier → Reranker → Policy. What changes is the *configuration* of each component: the prompts, the thresholds, the hints, the rules.

This is not a training pipeline. No models are fine-tuned. The classifier is Claude, called via API, with a system prompt that gets iteratively refined. The improvement loop is prompt optimization, not weight optimization.

This is a *configuration search* guided by structured evaluation. The artifacts make the search legible. The component graph makes the search targeted. The eval loop makes the search measurable. That's the architecture.

Three Agents, One Improvement Loop

The improvement loop is the pipeline. Everything else is configuration.

That statement sounds reductive until you see what “configuration” means in practice. In a traditional voice pipeline, configuration means a YAML file someone wrote six months ago. In this system, configuration means the output of three specialized agents — each with tool access, each producing typed artifacts, each operating on the failures of the previous iteration.

The agents are not autonomous in the science-fiction sense. They are autonomous in the engineering sense: given a well-defined objective and a set of tools, they execute a multi-step workflow without human intervention. The human defines the intents. The agents build, test, and harden the pipeline.

Agent 1: The Eval Curator

Role: Generate a diverse, structured evaluation dataset from an intent schema.

The Eval Curator takes the parsed `IntentSchemaArtifact` — a list of intents with definitions, examples, and disambiguation rules — and produces an `EvalDatasetArtifact` containing test utterances across three splits.

Why three splits matter:

Split	Purpose	Mutation Policy
train	Visible to optimization loop. The ETL agent sees these results.	Can be expanded with adversarial examples
dev	Validation set. Used to detect overfitting to train.	Can be expanded, never moved to train

Split	Purpose	Mutation Policy
holdout	Immutable. Never seen during optimization. Final integrity check.	Never modified, never expanded

The holdout split is sacred. If the system achieves 5% IER on train and dev but 20% IER on holdout, the optimization loop overfit. The holdout catches this.

What the curator generates:

For each intent, the curator produces utterances with explicit phenomenon tags:

```
EvalExample(
    example_id=new_artifact_id(),
    split="dev",
    source="curated",
    modality="text",
    utterance_text="uh can you check my balance",
    expected_intent="check_balance",
    phenomenon_tags=["filler_words"],
)
```

Phenomenon tags mark *what makes this example hard*:

- `direct` — Bare intent name as utterance (“check balance”)
- `natural_language` — Conversational framing (“I need help with checking my balance”)
- `question` — Question form (“Can you handle check balance for me?”)
- `polite` — Polite prefix (“please check my balance”)
- `filler_words` — Disfluency markers (“uh can you check my balance”)
- `out_of_domain` — Utterances that should map to fallback (“Tell me a joke”)
- `description_based` — Uses the intent description, not the name (“I’m calling about viewing my current account balance”)

The tags serve two purposes. First, they enable *phenomenon-level analysis* — if 80% of filler-word examples fail, the normalizer’s filler-stripping rules need work. Second, they guide the adversarial agent’s example generation — it generates more examples in the phenomenon categories that show the highest failure rates.

Coverage tracking:

```
coverage_summary = {
    split_name: sum(1 for example in examples if example.split == split_name)
    for split_name in ("train", "dev", "holdout")
}
```

The coverage summary is not a vanity metric. It’s a constraint checker. If holdout has zero examples for an intent, the system cannot validate that intent’s accuracy on unseen data. The summary surfaces gaps before the eval loop runs.

Agent 2: The ETL Pipeline Agent

Role: Run evaluations, analyze failures, rewrite pipeline configuration, and re-evaluate — in a tool-use loop.

The ETL Pipeline Agent is the workhorse. It has access to five tools, each defined as a JSON schema and executed against shared agent context:

```
store_evals      → Persist test cases to the evaluation set
run_eval_suite   → Execute all evals against current pipeline config, return IER
analyze_failures → Return failed cases with expected vs. actual intent
update_prompt_config → Rewrite the classifier system prompt, threshold, and ASR hints
classify_utterance → Spot-check a single utterance against current config
```

The agent operates in a loop. Here's the actual flow:

1. **Store evals** — The agent receives the curated eval set and stores it in context
2. **Run eval suite** — Every example is classified against the current pipeline config. The tool returns IER, correct/total counts, and up to 10 failure details
3. **Analyze failures** — The agent inspects which utterances were misclassified and why
4. **Update prompt config** — Based on failure analysis, the agent rewrites the system prompt. It may add disambiguation instructions, adjust the confidence threshold, or update ASR keyword hints
5. **Run eval suite again** — Measure the impact. Did IER improve?
6. **Repeat** until satisfied or iteration budget is exhausted

This is not a single LLM call. It's a multi-turn tool-use loop where the agent decides which tool to call next based on the results of the previous call. The agent is Claude, running via the Anthropic API with `tools=` parameter, making autonomous decisions about how to optimize the pipeline.

What the eval suite actually measures:

```
async def _exec_run_eval_suite(context):
    for ev in context.evals:
        intent_result = await extractor.extract(
            text=ev["utterance"],
            intent_prompt=context.intent_prompt,
            config=context.config,
        )
        results.append({
            "utterance": utterance,
            "expected": expected,
            "actual": intent_result.detected_intent,
            "confidence": intent_result.confidence,
            "correct": intent_result.detected_intent == expected,
        })

    ier = 1.0 - (correct / total)
```

Every utterance goes through the full pipeline — intent prompt + current config □ Claude API □ parsed result □ match against expected intent. IER is the percentage that got it wrong. This is the number the system is trying to minimize.

What “update prompt config” means concretely:

The agent rewrites three things: - `system_prompt` — The full system prompt sent to Claude for intent classification - `confidence_threshold` — The minimum confidence below which the system abstains - `asr_hints` — Keywords hinted to the ASR provider for better transcription

The system prompt is the biggest lever. When the agent sees that “cancel subscription” and “billing inquiry” are confused, it adds explicit disambiguation instructions to the prompt. When it sees that filler-word utterances are failing, it adds instructions to ignore disfluency markers. The prompt evolves through the loop based on empirical failure data, not human intuition.

Agent 3: The Adversarial Agent

Role: Cluster failures by confused intent pairs, generate targeted edge cases, and propose component-level fixes.

The Adversarial Agent is the red team. It takes the `EvaluationReportArtifact` from the most recent eval run and produces an `AdversarialFindingsArtifact` containing three things: failure clusters, proposed adversarial examples, and recommended component changes.

Failure clustering:

```
def _cluster_failures(self, report):
    grouped = defaultdict(list)
    for case in report.hard_cases:
        if not case.correct:
            grouped[(case.expected_intent, case.predicted_intent)].append(case.example_id)

    return [
        FailureCluster(
            title=f"{expected} confused with {predicted}",
            affected_intents=[expected, predicted],
            suspected_component_ids=["intent_classifier", "decision_policy"],
            example_ids=example_ids,
        )
        for (expected, predicted), example_ids in grouped.items()
    ]
```

The clustering is not random. It groups failures by *confused intent pairs* — “cancel_subscription confused with billing_inquiry” is a cluster. “check_balance confused with account_status” is a different cluster. Each cluster identifies the affected intents and the suspected component.

Adversarial example generation:

For each failed case, the agent generates variants designed to stress-test the same failure mode:

```
variants = [
    f"um {source_text}",          # filler word prefix
    f"can you please {source_text}", # polite framing
    f"{source_text} right now",    # urgency suffix
]
```

Each variant is tagged with `phenomenon_tags=["adversarial", "filler_words", "failure_driven"]` and annotated with `confusable_with=[predicted_intent]`. These examples are added to the eval set and the ETL agent re-optimizes — now with harder examples that target the known failure modes.

Component-level recommendations:

The adversarial agent doesn't just generate examples. It proposes specific fixes:

- *"Add failed-example keywords to ASR hints"* — When misclassification traces to transcription errors
- *"Expand classifier disambiguation rules for confused intent pairs"* — When the classifier prompt doesn't distinguish similar intents
- *"Lower the abstain threshold slightly for ambiguous but in-domain utterances"* — When the decision policy is too aggressive about falling back

These recommendations are stored in `recommended_component_changes` and inform the next ETL optimization cycle.

The Loop in Motion

Here's what a complete build looks like:

Iteration 0:

Eval Curator → 49 examples (7 per intent × 6 intents + 7 fallback)

ETL Agent → runs eval, IER = 0.22, rewrites prompt

ETL Agent → re-runs eval, IER = 0.14

Iteration 1:

IER 0.14 > target 0.08

Adversarial Agent → clusters 7 failures into 3 groups

Adversarial Agent → generates 21 adversarial examples

ETL Agent → re-optimizes with 70 examples, IER = 0.10

Iteration 2:

IER 0.10 > target 0.08

Adversarial Agent → clusters 7 failures into 2 groups

Adversarial Agent → generates 21 more adversarial examples

ETL Agent → re-optimizes with 91 examples, IER = 0.07

IER 0.07 ≤ target 0.08 → Pipeline ready

Three iterations. Zero human intervention. The pipeline went from 22% intent error rate to 7% by generating its own test cases, finding its own failures, and fixing its own configuration.

What Changes Operationally

Before this system: 1. Product manager writes intent descriptions 2. Engineer writes a classifier prompt 3. Engineer tests with 5-10 examples 4. Deploy and hope 5. Customer complaints surface failures weeks later 6. Engineer manually adjusts prompt, repeat from step 3

After this system: 1. Product manager writes intent descriptions via a prompt 2. System builds, tests, and hardens the pipeline automatically 3. Status polling shows build progress — eval curator complete, ETL optimizing, IER at 0.12... 4. Pipeline reaches target IER and becomes ready to invoke 5. Invoke uses the optimized config — the prompt, thresholds, and hints that survived the improvement loop

The human contribution is the *what* — which intents matter. The system handles the *how* — which prompt works, which threshold is right, which examples break it.

From Prompt-and-Pray to Continuous Improvement

Every voice pipeline in production today is exactly as good as the day it was deployed. This one gets better.

That's not a tagline. It's an architectural property. The system doesn't just classify intents — it generates evaluations, measures its own performance, finds its own failures, and iterates its own configuration. The pipeline *is* the improvement loop. Deployment is not the end of the process. It's the beginning.

What “Prompt-and-Pray” Actually Costs

The current state of voice pipeline engineering is roughly where web development was before automated testing. People build things, poke at them manually, convince themselves they work, and ship.

The process looks like this:

1. Write an intent classification prompt
2. Test it on the five utterances you can think of
3. They all work — ship it
4. A month later, customer complaints reveal that “reschedule my appointment” routes to “cancel appointment” 40% of the time
5. Fix the prompt for that case. Break two other cases in the process
6. Repeat until the next reorg

This is not engineering. This is folk medicine. And it persists not because engineers are careless, but because the infrastructure for doing it right didn't exist. There was no way to automatically generate comprehensive eval suites. There was no way to run them against the pipeline systematically. There was no way to analyze failures, propose fixes, and re-evaluate in a loop.

Now there is.

What Changes Operationally

Three shifts that matter:

No manual prompt tuning. The ETL Pipeline Agent rewrites the classifier system prompt based on empirical failure data. It sees which utterances are misclassified, which intents are confused, and what patterns the failures share. It rewrites the prompt to address those specific failure modes.

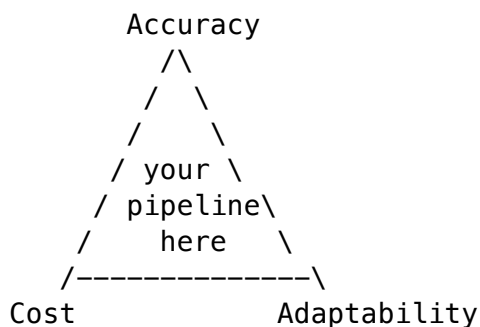
Then it re-evaluates to verify the fix worked. This is not “try a different prompt and see” — it’s measure-diagnose-fix-verify, automated.

No hand-curated test sets. The Eval Curator Agent generates diverse test utterances with phenomenon coverage — filler words, polite forms, question framing, out-of-domain, description-based. It splits them into train, dev, and immutable holdout. The adversarial agent extends the set with targeted edge cases that stress-test known failure modes. The eval set is comprehensive *because it was generated to be*, not because someone remembered to add an example for “uh can you check my balance.”

No “it works on my examples.” The holdout split exists specifically to catch overfitting. The system can optimize IER on train and dev all day — if holdout IER doesn’t improve, the optimization is memorizing patterns instead of learning them. The three-way split is the difference between “it works” and “it generalizes.”

The Tradeoff You’re Actually Navigating

Every voice pipeline makes implicit choices along three axes:



Accuracy: How often does the pipeline get the intent right? Measured by IER on holdout.

Cost: How many API calls, how much compute, how much latency per classification? Every improvement iteration costs Claude API calls. Every reranker step adds latency. Every additional eval example extends build time.

Adaptability: How quickly can the pipeline adjust to new intents, new utterance patterns, new failure modes? A hand-tuned prompt is cheap and accurate but brittle — it adapts slowly. A fully automated loop is expensive to run but adapts in minutes.

Most teams are pinned to the **Cost–Accuracy edge**: they hand-tune a prompt until it’s accurate enough on the examples they know about, and they keep costs low by not running evaluations. They sacrifice adaptability entirely.

This system lets you move along the surface deliberately:

- **High accuracy, high cost, high adaptability:** Run the full build loop with multiple adversarial iterations. IER drops to target. Build takes minutes and consumes API calls. New intents can be added and the loop re-runs.
- **Moderate accuracy, low cost, moderate adaptability:** Run a single eval-and-optimize pass. Skip adversarial iterations. Good enough for low-stakes applications.
- **High accuracy, moderate cost, moderate adaptability:** Set a tight IER target but cap iteration count. The system optimizes as far as it can within budget.

The `OptimizationObjective` in the build context controls where on this surface the system operates. The artifact architecture makes the tradeoff explicit — you can see exactly how many iterations it took, how many API calls were consumed, and what IER was achieved at each step.

What the Artifact Architecture Enables

The typed artifact chain is not just good engineering hygiene. It enables three capabilities that unstructured systems can't provide:

Reproducibility. Every build produces a chain of artifacts — `IntentSchemaArtifact` $\square$ `EvalDatasetArtifact` $\square$ `PipelineGraphArtifact` $\square$ `EvaluationReportArtifact` $\square$ `AdversarialFindingsArtifact`. Each artifact references its inputs via `ArtifactRef`. You can re-construct any build, re-run any evaluation, and diff any two pipeline configurations.

Debuggability. When IER regresses, the `EvaluationReportArtifact` contains per-intent accuracy, a full confusion matrix, and component-level traces for hard cases:

```
class EvalCaseResult(BaseModel):
    example_id: str
    expected_intent: str
    predicted_intent: str
    confidence: float
    correct: bool
    component_traces: list[ComponentTrace]
```

```
class ComponentTrace(BaseModel):
    component_id: str
    component_kind: str
    input_snapshot: dict[str, Any]
    output_snapshot: dict[str, Any]
    latency_ms: int
```

Each hard case carries a full trace — what went into each component, what came out, how long it took. This is attribution, not guessing.

Composability. The `PipelineGraphArtifact` separates component specs from the graph topology. You can swap the ASR provider without touching the classifier. You can add a reranker without changing the decision policy. You can version each component independently. The graph structure makes the pipeline modular in practice, not just in theory.

Who This Is For

This system is for teams that:

- Run voice pipelines where intent accuracy directly affects customer experience or revenue
- Have enough volume that a 5% IER improvement is worth automating
- Are tired of manually tuning classifier prompts and never knowing if they made things better or worse
- Want to add new intents without re-testing the entire pipeline by hand
- Need auditability — the ability to explain *why* the pipeline classifies a given utterance the way it does

This system is *not* for teams that:

- Have three intents and can test them manually in five minutes
- Don't have tolerance for build-time API costs (the improvement loop makes Claude calls)
- Need sub-100ms classification latency (the system uses LLM-based classification, not a lightweight model)

The Closing Insight

There is a category error embedded in most voice pipeline work. Teams treat the pipeline as a *static artifact* — something you build, configure, deploy, and maintain. The ASR is the pipeline. The classifier prompt is the pipeline. The confidence threshold is the pipeline.

But the ASR model will encounter utterance patterns it hasn't seen. The classifier prompt will face intent combinations it doesn't handle. The confidence threshold will be wrong for edge cases that haven't surfaced yet.

The pipeline isn't the ASR model or the classifier. The pipeline is the loop that improves them.

A static artifact degrades from the moment it's deployed, because the world changes and the artifact doesn't. A loop that evaluates, identifies failures, generates adversarial tests, and re-optimizes configuration — that's a system that *responds* to the world instead of assuming it stays fixed.

The question is not "how good is your voice pipeline today." The question is "how does your voice pipeline get better tomorrow."

This system is the answer: not a better model, but a better loop. Not a smarter prompt, but a system that writes, tests, and rewrites its own prompts until the failures stop. Not configuration — continuous improvement, encoded as architecture.